



1

Lab

Data Lab: Tính toán Bits (Bộ đề 2)

Data Lab: Manipulating Bits

Thực hành Lập trình Hệ thống

Lưu hành nội bộ

A. TỔNG QUAN

A.1 Mục tiêu

- Tìm hiểu các phép toán trên các bit nhị phân (**Bitwise Operation**):
 - **NOT** ($\sim$),
 - **AND** ($\&$),
 - **OR** ($\mid$),
 - **XOR** ($\wedge$),
 - **Left Shift** ($\ll$), **Right Shift** ($\gg$)
- Nắm vững các kiến thức về lập trình tương tác bit nhị phân trên C
- Thực hành giải các bài toán về bit nhị phân
- Hiểu biểu diễn số nguyên bù 2, hiện tượng tràn số (overflow) và hành vi không xác định (undefined behavior) trong C
- Hiểu biểu diễn số thực dấu phẩy động theo chuẩn IEEE-754 và thao tác trên mức bit

A.2 Môi trường

- Hệ điều hành Linux (hoặc WSL trên Windows) với trình biên dịch gcc.
- Lệnh biên dịch dùng khi chấm: `gcc -O1 -Wall -fwrapv datalab.c -o datalab (-fwrapv quy định phép cộng số có dấu bị tràn sẽ quay vòng theo bù 2, đúng như giả định của bài lab). Có thể biên dịch thêm với -fsanitize=undefined để xem những chỗ lỗi giải gây tràn số hoặc dịch bit không hợp lệ.`
- Có thể soạn code bằng IDE bất kỳ, nhưng kết quả được chấm trên gcc/Linux.

A.3 Liên quan

- Sinh viên cần vận dụng kiến thức trong Chương 2 (Lý thuyết): số nguyên (CSAPP mục 2.1–2.3) và số thực dấu phẩy động (CSAPP mục 2.4).
- Các kiến thức này đã được giới thiệu trong nội dung lý thuyết đã học do đó sẽ không được trình bày lại trong nội dung thực hành này.
- Xem Mục B (Kiến thức nền & công cụ hỗ trợ) trước khi làm bài. Tài liệu tham khảo ở Mục E.

B. KIẾN THỨC NỀN & CÔNG CỤ HỖ TRỢ

B.1 Bảng chân trị

Dưới đây là bảng chân trị tham khảo cho 2 giá trị X và Y với các phép toán Bitwise

x	y	x & y	x y	x ^ y	~x
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Trong bài lab này, chỉ có toán tử **!** trả về dưới dạng 2 giá trị true (1) hoặc false (0), có thể sử dụng trong các câu đố có trả về 1 (True) hoặc 0 (False).

B.2 Hàm hỗ trợ: PrintBits & PrintBitsOfByte

Dưới đây là các hàm trung gian có thể giúp chúng ta quan sát rõ hơn giá trị của 1 số biểu diễn dưới dạng nhị phân:

- **Hàm PrintBits:** biểu diễn 1 số dưới dạng nhị phân dưới dạng đầy đủ 32 bit

```
void PrintBits(unsigned int x) {
    int i;
    for (i = 8 * sizeof(x)-1; i >= 0; i--) {
        (x & (1 << i)) ? putchar('1') : putchar('0');
    }
    printf("\n");
}
```

Ví dụ: **PrintBits(16)** == 0000 0000 0000 0000 0000 0000 0001 0000

- **Hàm PrintBitsOfByte:** biểu diễn 1 số dưới dạng nhị phân có 8 ký tự (chỉ lấy 8 bit thấp)

```
void PrintBitsOfByte(unsigned int x) {
    int i;
    for (i = 7; i >= 0; i--) {
        (x & (1 << i)) ? putchar('1') : putchar('0');
    }
    printf("\n");
}
```

Ví dụ: **PrintBitsOfByte(16)** == 0001 0000

B.3 Một số phép tính hữu ích

Lưu ý: các phép tính dưới đây áp dụng cho số biểu diễn bằng **32 bit**.

	Phương pháp	Phép tính	Ví dụ
Xét dấu của một số x	Xét giá trị bit có trọng số cao nhất	$x \gg 31$ hoặc $(x \gg 31) \& 1$	- $(x \gg 31) = 0$ hoặc $(x \gg 31) \& 1 = 0$ thì $x \geq 0$ - $(x \gg 31) = 0xFFFFFFFF$ hoặc $(x \gg 31) \& 1 = 1$ thì $x < 0$
Giữ lại một số bit nhất định, còn lại bỏ qua của số x	- Các bit cần lấy được & với bit 1 để giữ nguyên giá trị của nó - Các bit bị bỏ sẽ được & với bit 0	- Tạo một "mask" có bit 1 và 0 tương ứng với vị trí bit cần lấy và bỏ. - Sử dụng toán tử & số x và "mask"	Lấy 4 bit thấp nhất của x. - Mask: 0000 ... 0000 0000 1111 = 0x0000000F - $x \& \text{mask}$ = kết quả

B.4 Số thực dấu phẩy động IEEE-754 (dùng cho Phần 3)

Một số float (32 bit) gồm 3 trường:

- **Bit 31: s – bit dấu (0 dương, 1 âm).**
- **Bit 30–23: exp – 8 bit phần mũ, bias = 127.**
- **Bit 22–0: frac – 23 bit phần định trị.**

Tách các trường từ mẫu bit uf: $s = uf \gg 31$; $exp = (uf \gg 23) \& 0xFF$; $frac = uf \& 0x7FFFFFFF$.

Giá trị được diễn giải theo 4 trường hợp của exp:

- **Chuẩn hóa (exp khác 0 và khác 0xFF):** $V = (-1)^s \times 1.\text{frac} \times 2^{(exp - 127)}$.
- **Không chuẩn hóa (exp = 0):** $V = (-1)^s \times 0.\text{frac} \times 2^{(-126)}$. Bao gồm +0 (0x00000000) và -0 (0x80000000).
- **Vô cực (exp = 0xFF, frac = 0):** $+\infty = 0x7F800000$, $-\infty = 0xFF800000$.
- **NaN (exp = 0xFF, frac khác 0), ví dụ 0xFC000000.**

Ví dụ: $1.0 = 0x3F800000$; $1.5 = 0x3FC00000$; $-2.5 = 0xC0200000$.

Hàm hỗ trợ chuyển đổi giữa float và mẫu bit (chỉ dùng trong hàm main để kiểm tra, KHÔNG dùng trong lời giải; cần `#include <string.h>`):

```
unsigned f2u(float f) { unsigned u; memcpy(&u, &f, sizeof u); return u; }
float u2f(unsigned u) { float f; memcpy(&f, &u, sizeof f); return f; }
```

Tham khảo chi tiết: CSAPP mục 2.4.2–2.4.3.

C. THỰC HÀNH

C.1 Yêu cầu chung

- **Viết các hàm bằng ngôn ngữ C, đặt chung trong 1 file datalab.c.**
Bài lab gồm 3 phần, tổng điểm tối đa 20: Phần 1 và Phần 2 (số nguyên, 14 điểm) làm TẠI LỚP; Phần 3 (số thực, 6 điểm) làm TẠI NHÀ.
- Luật lập trình cho Phần 1 và Phần 2 – sinh viên chỉ được sử dụng các toán tử gồm:
 - Các toán tử trên bit gồm: **~, &, ^, |, <<, >>**
 - Một số toán tử khác được phép: **!, + và = (gán)**
 - **Không được** dùng các toán tử khác với các toán tử ở trên, ví dụ: **&&, ||, -, ==, !=, *, /, %**
 - **Không được** dùng các phép lập trình cấu trúc (control structures) như **loop, conditionals** (if/else và if inline), **switch,...**
 - Chỉ dùng hằng số từ 0x00 đến 0xFF (0–255). Hằng lớn hơn phải tự tạo, ví dụ 0xFF << 24.
 - Giả định: int 32 bit, biểu diễn bù 2; phép >> trên int là dịch số học. Không dịch với số lượng âm hoặc ≥ 32 (hành vi không xác định trong C).
- **Luật lập trình cho Phần 3 (số thực):**
 - Tham số hoặc giá trị trả về đại diện cho số float có kiểu unsigned, chứa mẫu bit IEEE-754 của số đó (xem mục B.4); tham số hoặc giá trị trả về là số nguyên có kiểu int.
 - Được dùng mọi phép toán trên số nguyên (kể cả -, ==, <, &&, ||), hằng số bất kỳ và cấu trúc điều khiển (if/else, vòng lặp).
 - Không được dùng kiểu float/double, hằng số thực, union, ép kiểu sang số thực hay phép toán số thực.
- Mỗi câu đố bao gồm 4 nội dung chính:
 - **Tên hàm:** Tên của hàm và kiểu giá trị trả về chứa nội dung lời giải.
 - **Yêu cầu thực thi:** Yêu cầu cần đạt được khi giải câu đố, trong đó bao gồm giải thích ý nghĩa của hàm và các phép thử mẫu dùng cho việc đánh giá điểm số (với PT là viết tắt của *Phép Thử*).
 - **Điểm đánh giá:** Mức điểm đánh giá cho mỗi câu đố, mức điểm cao hơn cho biết câu đố đó khó hơn.

- **Max Ops: Số lượng toán tử tối đa được phép sử dụng. Mỗi toán tử tính 1 lần; phép gán (=) và hằng số không tính. Nếu kết quả đúng nhưng vượt quá số lượng này, điểm câu đó bị chia đôi.**
- Kiểm tra kết quả: sinh viên tự viết hàm main chạy các phép thử (PT) trong đề và bổ sung các giá trị biên 0, -1, 0x7FFFFFFF (INT_MAX), 0x80000000 (INT_MIN). GVTH chấm bằng bộ kiểm thử riêng, gồm cả test ngẫu nhiên.

C.2 Bài tập

C.2.1 Tính toán bit đơn giản (làm tại lớp)

Các hàm này có đầu vào là các số nguyên 32 bit và đầu ra cũng là các số nguyên **32 bit** được tính toán bằng các toán tử trên bit.

- Số câu đố: 6
- Điểm tối đa: 7

Viết các hàm với các yêu cầu như sau:

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
-----	---------	------------------	------	---------

1.1	int subtract(x,y)	Tính giá trị $x - y$ không dùng dấu $-$. Input: Số nguyên x, y PT: $\text{subtract}(10,3) == 7$ && $\text{subtract}(0,5) == -5$ && $\text{subtract}(-4,-9) == 5$	0.5	8
1.2	int cal100x(x)	Tính giá trị $100 * x$ mà không được dùng phép nhân. Input: Số nguyên x (giới hạn sao cho $100x$ biểu diễn đủ trong số có dấu 4 byte) PT: $\text{cal100x}(0) == 0$ && $\text{cal100x}(7) == 700$ && $\text{cal100x}(-3) == -300$	1	10
1.3	int getNibble(x,n)	Trả về nibble (nhóm 4 bit) thứ n của x (các nibble được đánh thứ tự từ 0 đến 7 từ phải sang trái). Input: Số x nguyên; Số n ($0 \leq n \leq 7$) PT: $\text{getNibble}(0x12345678,0) == 0x8$ && $\text{getNibble}(0x12345678,5) == 0x3$ && $\text{getNibble}(0xABCDEF01,7) == 0xA$	1	10
1.4	int clearByte(x,n)	Xóa (đặt về 0) toàn bộ 8 bit của byte thứ n của số nguyên x, các byte khác giữ nguyên (các byte được đánh thứ tự từ 0 đến 3 từ phải sang trái). Input: số nguyên x, số n ($0 \leq n \leq 3$) PT: $\text{clearByte}(0x12345678,1) == 0x12340078$ && $\text{clearByte}(-1,0) == 0xFFFFFFFF00$ && $\text{clearByte}(0x5501,0) == 0x5500$	1.5	20
1.5	int mulpw2(x,n)	Tính kết quả $x \cdot 2^n$ với n âm (tức là chia x cho 2^{-n}), không dùng phép nhân, chia. Input: Số x nguyên dương; Số n âm ($-31 \leq n \leq -1$) PT: $\text{mulpw2}(20,-1) == 10$ && $\text{mulpw2}(60,-2) == 15$ && $\text{mulpw2}(32,-4) == 2$	1.5	8

1.6	int remPw2(x,n)	Tính phần dư $x \% 2^n$ với x là số nguyên có dấu (có thể âm); kết quả cùng dấu với x giống phép % của C. Input: Số x nguyên; Số n ($0 \leq n \leq 30$) PT: $\text{remPw2}(15,2) == 3 \ \&\& \ \text{remPw2}(-15,2) == -3$ $\&\& \ \text{remPw2}(-16,3) == 0 \ \&\& \ \text{remPw2}(-1,4) == -1$ Gợi ý: tính thương $q = x/2^n$ làm tròn về 0 bằng cách cộng độ lệch (bias) khi x âm (CSAPP mục 2.3.7), rồi lấy $x - q \cdot 2^n$.	1.5	20
-----	-----------------	--	-----	----

C.2.2 Các phép kiểm tra dựa trên tính toán bit (làm tại lớp)

Các hàm này có đầu vào là các số nguyên 32 bit và đầu ra là các giá trị đại diện cho **True/False**, tức là **0 hoặc 1 (1 bit)**, của phép đánh giá nào đó.

- Số câu đố: 6
- Điểm tối đa: 7

Viết các hàm với các yêu cầu như sau:

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
2.1	int isNegation(x,y)	Kiểm tra y có phải là số đối của x hay không ($y = -x$ theo toán học). Trả về 1 nếu đúng, 0 nếu sai. Lưu ý trường hợp số 0 (số đối của 0 là 0). Input: Số nguyên x, y PT: $\text{isNegation}(4,-4) == 1 \ \&\& \ \text{isNegation}(0,0) == 1 \ \&\& \ \text{isNegation}(5,-6) == 0 \ \&\& \ \text{isNegation}(3,3) == 0$ Báo cáo: giải thích kết quả của hàm với $x = y = 0x80000000$ (INT_MIN).	1.5	8
2.2	int isMul8Not16(x)	Kiểm tra số nguyên x có chia hết cho 8 nhưng không chia hết cho 16 hay không. Trả về 1 nếu đúng, 0 nếu sai. Input: Số nguyên x không âm PT: $\text{isMul8Not16}(8) == 1 \ \&\& \ \text{isMul8Not16}(24) == 1 \ \&\& \ \text{isMul8Not16}(16) == 0 \ \&\& \ \text{isMul8Not16}(12) == 0$	0.5	10
2.3	int isNonPositive(x)	Trả về 1 nếu x không dương ($x \leq 0$) Input: Số nguyên x PT: $\text{isNonPositive}(10) == 0 \ \&\& \ \text{isNonPositive}(-5) == 1 \ \&\& \ \text{isNonPositive}(0) == 1$	1	15

2.4	int isTopBit(x,n)	Trả về 1 nếu bit 1 cao nhất của x nằm đúng ở vị trí n, tức $2^n \leq x < 2^{n+1}$. Input: Số x nguyên dương; Số n ($0 \leq n \leq 30$) PT: isTopBit(5,2) == 1 && isTopBit(8,2) == 0 && isTopBit(7,3) == 0 && isTopBit(1,0) == 1	1	20
2.5	int addOK(x,y)	Trả về 1 nếu phép tính $x + y$ không bị tràn số (overflow) trên số nguyên 32 bit có dấu, ngược lại trả về 0. Input: Số nguyên x, y PT: addOK(3,4) == 1 && addOK(0x7FFFFFFF,1) == 0 && addOK(0x80000000,-1) == 0 && addOK(0x80000000,0x7FFFFFFF) == 1 && addOK(-1,-1) == 1 Gợi ý: tràn số chỉ xảy ra khi x và y cùng dấu, và dấu của tổng khác dấu của x.	1.5	20
2.6	int allOddBits(x)	Trả về 1 nếu tất cả các bit ở vị trí lẻ (bit 1, 3, 5, ..., 31) của x đều bằng 1, ngược lại trả về 0. Input: Số nguyên x PT: allOddBits(0xFFFFFDD) == 0 && allOddBits(0xAAAAAAAA) == 1 && allOddBits(0xFFFFFFFF) == 1 && allOddBits(0x2AAAAAAAA) == 0 Gợi ý: hằng số tối đa là 0xFF, cần tự tạo mask 0xAAAAAAAA từ 0xAA; hoặc gấp đôi x bằng phép & (16, 8 bit).	1.5	20

C.2.3 Ví dụ giải mẫu: câu 2.3 isNonPositive(x)

- **Yêu cầu:** Trả về 1 (True) nếu $x \leq 0$. Max Ops là 15.
- **Giải:** với x đầu vào có thể có 3 trường hợp tương ứng với các đầu ra tương ứng:

x	Output
<0	1
= 0	1
>0	0

Ta thấy cần phân biệt các trường hợp sau:

- o **x âm hay dương:** dựa vào bit có trọng số cao nhất (tham khảo mục B.3) xét $(x \gg 31)$ hoặc $(x \gg 31) \& 1$. Ở đây để xét $(x \gg 31)$ có giá trị hay không, có thể dùng $!(x \gg 31)$.
- o **x có bằng 0 hay không:** dùng $!x$.

Với 2 toán hạng cần xét là $!(x \gg 31)$ và $!x$, sinh viên thử tìm tiếp các phép toán cần sử dụng để kết hợp chúng ra output mong muốn?

x	!(x>>31)	!x	Output
< 0	0	0	1
= 0	1	1	1
> 0	1	0	0

Lưu ý: vì các hàm ở phần 2 yêu cầu chỉ đưa ra giá trị 0 hoặc 1 đại diện cho False hoặc True, kết quả cuối cùng có thể dùng phép **&** với **0x1** để chỉ lấy 1 bit thấp nhất.

C.2.4 Số thực dấu phẩy động (làm tại nhà)

Các hàm này làm việc trực tiếp trên mẫu bit (kiểu unsigned) của số float, phải xử lý đúng các trường hợp đặc biệt: ± 0 , số không chuẩn hóa, $\pm\infty$ và NaN. Tuân theo luật lập trình cho Phần 3 ở mục Yêu cầu chung và kiến thức ở mục B.4.

- Số câu đố: 4
- Điểm tối đa: 6
- Hình thức: làm tại nhà, nộp cùng thư mục bài theo thời hạn trên website môn học.

Viết các hàm với các yêu cầu như sau:

STT	Tên hàm	Yêu cầu thực thi	Điểm	Max Ops
3.1	unsigned float_nabs(uf)	Trả về mẫu bit của $- f $. Nếu f là NaN thì trả về nguyên uf. Input: mẫu bit uf của số float f PT: float_nabs(0x3F800000) == 0xBF800000 && float_nabs(0xC0200000) == 0xC0200000 && float_nabs(0x00000000) == 0x80000000 && float_nabs(0x7FC00000) == 0x7FC00000	1	10
3.2	int float_sign(uf)	Trả về 1 nếu $f > 0$, -1 nếu $f < 0$, và 0 nếu f là +0, -0 hoặc NaN. Input: mẫu bit uf của số float f PT: float_sign(0x3F800000) == 1 && float_sign(0xC0200000) == -1 && float_sign(0x80000000) == 0 && float_sign(0x7FC00000) == 0 && float_sign(0x00000001) == 1	1	15

3.3	unsigned float_pwr2(x)	Trả về mẫu bit của số float 2.0^x. Nếu kết quả nhỏ hơn số không chuẩn hóa nhỏ nhất thì trả về 0; nếu quá lớn thì trả về +∞. Input: Số nguyên x PT: float_pwr2(0) == 0x3F800000 && float_pwr2(-1) == 0x3F000000 && float_pwr2(-127) == 0x00400000 (số không chuẩn hóa) && float_pwr2(-149) == 0x00000001 && float_pwr2(-150) == 0 && float_pwr2(128) == 0x7F800000 (+∞) Gợi ý: chia x thành 4 khoảng: quá nhỏ, không chuẩn hóa, chuẩn hóa, quá lớn (CSAPP bài tập 2.90).	2	30
3.4	int float_isInt(uf)	Trả về 1 nếu f là một số nguyên (kể cả +0 và -0); trả về 0 nếu f có phần thập phân khác 0, hoặc f là NaN, ±∞. Input: mẫu bit uf của số float f PT: float_isInt(0x40400000) == 1 (3.0) && float_isInt(0x3FC00000) == 0 (1.5) && float_isInt(0x80000000) == 1 (-0) && float_isInt(0x3F000000) == 0 (0.5) && float_isInt(0x4B000001) == 1 (8388609.0) && float_isInt(0x7F800000) == 0 (+∞)	2	30

D. YÊU CẦU & ĐÁNH GIÁ

Sinh viên thực hiện bài thực hành theo **nhóm (tối đa 3 SV)**, theo 2 hình thức:

- Tại lớp: demo kết quả Phần 1 và Phần 2 và báo cáo tiến độ với GVTH.
- Nộp file code (datalab.c, gồm cả Phần 3 làm tại nhà) và báo cáo tóm lược tại website môn học theo thời gian quy định.
- Báo cáo tóm lược cần có:
 - o Bảng kết quả từng câu: đúng/sai và số toán tử đã dùng.
 - o Giải thích trường hợp $x = y = 0x80000000$ ở câu 2.1 (isNegation).
 - o Giải thích cách xử lý khoảng số không chuẩn hóa ở câu 3.3 (x từ -149 đến -127) và trường hợp số mũ $E \geq 23$ ở câu 3.4.
- Tiêu chí chấm mỗi câu: đúng với toàn bộ bộ kiểm thử của GVTH; dùng toán tử hoặc cấu trúc bị cấm thì không được điểm câu đó; vượt Max Ops thì bị chia đôi điểm.

Đặt tên thư mục nộp bài theo quy tắc sau:

Lab1-NhomX-MSSV1-MSSV2-MSSV3

Ví dụ: *Lab1-Nhom2-24520901-24520111-24521616*

E. THAM KHẢO

[1] Basics of Bit Manipulation [Online] Available at:

<https://www.hackerearth.com/fr/practice/basic-programming/bit-manipulation/basics-of-bit-manipulation/tutorial/>

[2] Randal E. Bryant, David R. O'Hallaron (2011). *Computer System: A Programmer's Perspective (CSAPP)*

[3] CSAPP: mục 2.1.7–2.1.10 (phép toán bit, logic, dịch), 2.3.7 (chia cho lũy thừa của 2), 2.4 (dấu phẩy động); bài tập 2.64, 2.73, 2.90, 2.92.

[4] W. Dietz, P. Li, J. Regehr, V. Adve (2012). Understanding Integer Overflow in C/C++. ICSE 2012.

HẾT